

アピールポイント

ロボット掃除機+見守りカメラ シミュレーション

本ドキュメントでは、本シミュレーションにおいて特に工夫した5つの点を、設計の根拠・数式・実測結果とともに述べる。実装は役割ごとに分離した以下の構成を採る。

モジュール	役割
MotionProfiler	台形（区間長によっては三角形に縮退する）速度プロファイルの生成
TrajectoryFollower	状態フィードバック+PID+フィードフォワードによる目標速度・角速度の追従、および合成加速度のハードクランプ
WheelDrive	並進力・ヨーモーメントを左右仮想輪への <code>AddForceAtPosition</code> に分配
AccelerationLogger / DataVisualizer	カメラ頂部の水平加速度・ジャークの記録とUnity実行中のリアルタイム表示
CourseBuilder / ValidityChecker	矩形データからの壁・経路の自動生成、無効判定

アピールポイント1：合成加速度の上限を制御則によって物理的に保証した

PIDの追従精度に依存させず、力の計算段階でカメラ頂部の水平合成加速度が1.00 m/s²を 超え得ない構造にした。

カメラ頂部の水平合成加速度は、ロボット幾何（カメラが重心の真上、かつ水平面内のヨー回転のみ）から 重心の並進加速度と一致する（ $a \times r = 0$, $\omega \times (\omega \times r) = 0$ ）。この事実に基づき、フィードフォワード加速度と PID 補正加速度の合計値を毎ステップ0.98 m/s²でハードクランプする実装とした。台形速度プロファイルは 巡航フェーズと減速フェーズの境界で目標加速度が不連続に変化する（ジャーク不連続）が、この過渡応答を含め PID追従誤差の大小に関わらず合成加速度が上限を超えないことを、力の計算段階で直接保証している。実測（0.01秒刻み、1706ステップ、17.05秒間）で上限超過は0件であった。

アピールポイント2：状態フィードバックにより横方向のズレを高精度に補正した

ヘディング誤差だけでなく横方向の位置誤差も状態として扱うことで、通路幅600mm・ロボット直径300mm（片側クリアランス150mm）という狭いコースでも走行中の横方向誤差を 常に2mm以内に抑えた。

直進区間の姿勢保持は、ヘディング誤差のみを見るP制御ではなく、状態ベクトル $x = [\text{横方向位置誤差 } y, \text{ヘディング誤差 } \theta]$ に対する線形状態フィードバック

$$\omega = -K_{lat} \cdot y + K_{head} \cdot \theta$$

として設計した。運動学 $dy/dt = -v \cdot \theta$ 、 $d\theta/dt = -\omega$ からこの制御則による 閉ループ系を導くと

$$y'' + K_{head} \cdot y' + v \cdot K_{lat} \cdot y = 0$$

という2階線形微分方程式になる。これは質量ばねダンパ系と同形であり、 $K_{lat} \cdot K_{head}$ をいずれも 正に取れば、任意の速度 v に対して安定に収束することが式の形から保証される。単純なヘディング保持では 横方向

の蓄積誤差を直接補正する経路がないのに対し、この設計では横方向誤差そのものを状態として フィードバックしている。

アピールポイント3：速度上限を撤廃し、最短時間軌道に自動的に縮退させた

台形速度プロファイルの性質を利用して速度上限そのものを実質無制限にし、各区間ごとに 加速度上限で決まる理論上最短の速度プロファイルを自動生成することで、走破タイムを 23.9秒から15.5秒へ短縮した。

課題の必須条件は「駆動輪の最大トルクによる制約は考慮しなくてよい。走行速度の上限も設けない」である。台形速度プロファイルにおいて最高速度に到達するために必要な加減速距離が区間長を上回る場合、プロファイルは自動的に三角形（加速度上限で加速し続け、区間中央で切り返して減速する軌道）に縮退する。速度上限を実質無制限（100 m/s）に設定することでこの性質を利用し、各区間について

$$v_{\text{peak}} = \sqrt{(\text{区間距離} \times \text{加速度上限})}$$

で決まる、その区間で理論上最短の時間となる速度プロファイルを自動的に得ている。本コースの5直進区間（各1.2～1.8m）では、速度上限を0.5 m/sに固定していた初期実装（走破時間23.9秒）に対し、この変更のみで15.5秒まで短縮された。アピールポイント1の合成加速度ハードクランプと組み合わせることで、「制約ぎりぎりまで攻めた最短時間」と「制約の絶対遵守」を両立している。

アピールポイント4：4件の異常挙動すべてを運動方程式に基づいて原因特定・解決した

「なぜか動かない」「なぜか暴走する」といった現象を試行錯誤で誤魔化さず、力学の計算に立ち返って根本原因を特定し、対症療法ではなく構造的な修正を行った。特に4.1は、剛体力学の外積計算を手計算で追跡して発見したもので、この1箇所の修正だけで複数の異常現象が同時に解消している。

4.1 車輪力配分の符号誤り

初期実装では、旋回を開始するたびにロボットが単調に加速回転し、Unity Rigidbodyの角速度上限（既定7 rad/s）に達したまま戻らないという現象が生じた。左右仮想輪に加わる力とその作用点から生じる正味トルクを

$$\tau = r_{\text{left}} \times F_{\text{left}} + r_{\text{right}} \times F_{\text{right}}$$

として展開すると、 $r_{\text{right}} \times \text{forward} = -up$ （Unityの左手系）であることから、右輪の推進力を増やすほど実際に生じるヨートルクは指令した符号と逆方向になることが判明した。この結果、PID補正が誤差を減少させるどころか増幅する正のフィードバックとして働いていた。左右輪への力配分の符号を入れ替えることで、この一箇所の修正により暴走・振動・角速度飽和のすべてが同時に解消した。

4.2 床コライダー摩擦とアクチュエータ設計の整合性

床・車体コライダーの摩擦係数を1.0に設定した状態では、ロボットに推進力を加えても一切移動しないという現象が生じた。原因は、最大静止摩擦力 μmg （ $\mu=1.0$ の場合 約98 N）が、本設計の合成加速度ハードクランプにより理論上の上限が9.8 N（10 kg × 0.98 m/s²）に制限されている 駆動力を常に上回っていたことである。本設計では車輪の回転運動を個別にシミュレートせず、駆動輪位置に推進力を直接与える方式を採用しているため、Unity標準の床-車体間摩擦（滑り抵抗）を別途持たせると、実際には存在しない「車体が床を引きずられる」抵抗を二重に計上することになる。課題仕様の「転がり抵抗・軸受損失・空気抵抗などの軽微な損失は無視してよい」という規定に基づき、床コライダーの摩擦係数はゼロとした。駆動に必要なグリップは、車輪位置への力の直接付与というアクチュエータモデルの中に既に織り込まれている。

4.3 駆動力の作用高さとピッチ結合

駆動力を床面（重心から0.5 m下）に作用させる実装では、加減速のたびにピッチ方向の転倒トルクが 発生し、「水平面内のヨー回転のみ」という、カメラ加速度 = 重心並進加速度という等式成立の前提が 崩れることが確認された。駆動力の作用点を重心と同じ高さに変更することで、この結合を構造的に 排除した（左右対称な力の水平成分のみが残り、鉛直方向のモーメントアームがゼロになる）。

4.4 非ホロノミック拘束の欠落

4.2の摩擦をゼロにした結果、車輪が本来持つ「前後方向には転がるが横方向には滑らない」という 非ホロノミック拘束も同時に失われ、超信地旋回中にロボットが横方向へ緩やかに流れる現象が生じた。これに対し、車体の右方向（transform.right）成分の速度をFixedUpdate毎に明示的に ゼロへ落とす拘束を追加し、車輪グリップによる横滑り防止を近似的に再現した。

アピールポイント5：コース形状を座標のハードコードなしに汎用アルゴリズムで生成した

コース形状を5個の矩形データとして与えるだけで、壁・中心線経路・スタート/ゴール位置の すべてを自動導出する設計にし、コース変更に対する拡張性を持たせた。

矩形データからの境界座標の抽出、内外判定グリッドの構築、壁セグメントの生成、スタートセルからゴールセルまでの4連結BFS探索による中心線経路の導出、方向転換点でのウェイポイント抽出までを汎用アルゴリズムとして実装した。個々の壁や通路の座標を直接 書き込む実装ではないため、コース形状の変更は矩形データの書き換えのみで完結する。

参考：無効判定の実装

条件	実装
壁への接触	OnCollisionEnter + Wallタグ判定（Continuous Collision Detectionと併用）
コース外への移動	矩形データに基づく点の内外判定（壁貫通が生じた場合のフォールバックとしても機能）
転倒	ロボットのUp軸とワールドUp軸のなす角が閾値（既定15°）を超えた場合
計測終了タイミング	課題仕様「ゴールのラインを超えて完全に離れた時点」に合わせ、 OnTriggerEnter ではなく OnTriggerExit で判定